

Numerical Design Optimization Project 3 Report

6687

November 11 2025

Executive Summary

This project concerns the optimization of the Tilt-A-Whirl theme park ride so as to maximize “fun”. The Tilt-A-Whirl consists of several cars rotating on an uneven platform, where each car is free to spin about its own axis. As the cars travel around the uneven track, they spin unpredictably, which is a source of rider enjoyment. Thus, increasing chaos increases rider enjoyment. Therefore, the goal of this optimization was to maximize the standard deviation of angular velocity of the cars. Specifically, a 200% increase of the standard deviation of angular velocity over a nominal value.

The dynamics of the car were developed by R.L. Kautz and Bret M. Huggard [2], and suggest that the angular velocity of the track ω , the angle of the track above horizontal α_1 , and the size of each car r_2 , are the design variables, with standard deviation σ of angular velocity ϕ being the objective. The nominal design was $\omega = 6.5 * \frac{2\pi}{60} \frac{rad}{s}$, $\alpha_1 = 0.058rad$, $r_2 = 0.8m$, and $\sigma = 1.5203 \frac{rad}{s}$.

The dynamics of a car was simulated using Matlab’s ode45 numeric solver. From here, the standard deviation of angular velocity over that span of time was calculated for a range of design variables and a surrogate model was produced. Finally, optimization was performed on the surrogate model to determine the ideal design variables.

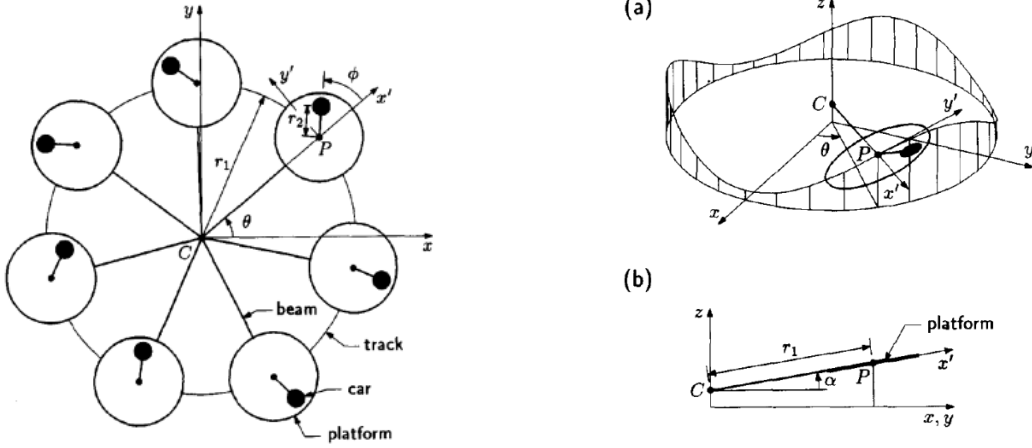


Figure 1: Tilt-A-Whirl Schematics [1]

Matlab's GPML library was used to create the surrogate model with Latin hypercube sampling to sample the design space. Fmincon was then used to perform the optimization on the surrogate. This optimization proved to be a success, with over 300% improvement over the nominal value of standard deviation of angular velocity with no violations of design variable constraints. However it should be noted that calculated standard deviation is likely an overestimate due to assumptions made during modeling.

Methodology

The first step in the design process was to develop a model of the car's dynamics. An in depth derivation of the dynamics was performed by R.L. Kautz and Bret M. Huggard [2], and their resulting ODE is given as Equation 1.

$$\frac{d^2\phi}{d\tau^2} + \left(\frac{\gamma}{Q_0}\right)\frac{d\phi}{d\tau} + (\epsilon - \gamma^2\alpha)\sin(\phi) + \gamma^2\beta\cos\phi = 0 \quad 1$$

Where $\tau = 3\omega t$, $\gamma = \frac{1}{3\omega}\sqrt{\frac{g}{r_2}}$, $Q_0 = 20$, $\epsilon = \frac{r_1}{9r_2}$, $r_1 = 4.3$, $\alpha = \alpha_0 - \alpha_1\cos(\tau)$, $\alpha_0 = 0.036$, and $\beta = 3\alpha_1\sin(\tau)$. In order to create this model, several important assumptions had to be taken. First, and most importantly, it is assumed that a large standard deviation in

angular velocity means a more enjoyable ride. Then, each car is a point mass, all components are rigid, and each car's motion is independent of one another. Of these assumptions, the dominating assumptions are that standard deviation in angular velocity is correlated with fun, and that each car is a point mass. Increasing standard deviation of angular velocity too much could make the ride disorientating rather than fun, or at least unfit for small children who are assumed to be the target audience of the ride. Additionally, assuming each car is a point mass neglects the rotational inertia of a car loaded with passengers, which serves to underestimate the angular velocity of the car. Although these assumptions may augment reality, they do enable us to perform an optimization, and therefore must simply be acknowledged and taken into account when interpreting the final result.

The design variables were constrained as follows to ensure a feasible and manufacturable design.

$$3rpm \leq \omega \leq 8rpm \quad 2$$

$$0.1m \leq r_2 \leq 1.5m \quad 3$$

$$0rad \leq \alpha_1 \leq 0.3rad \quad 4$$

The parameterization of this problem chiefly concerns the use of a surrogate model. The decision to optimize a surrogate rather than optimizing the car dynamics directly, was taken since our goal was to maximize the standard deviation of angular velocity rather than angular velocity directly, which introduced randomness into our design space. Using a surrogate model comes with advantages and disadvantages; it allows us to perform a gradient based optimization, but our optimization is only as accurate as the surrogate. The surrogate's accuracy is dependent on two main factors: the length of time allowed to solve the ODE, and the resolution of the

sampling of the design space. Solving the ODE over larger spans of time allows for a more accurate calculation of the standard deviation of angular velocity, and a finer sampling of the design space allows for a tighter fit of the surrogate model. More accurate standard deviation calculations and finer sampling increases computational costs, however, and a reasonable balance must be found.

Matlab's ode45 method was used to solve Equation 1. The surrogate model was created with the options and hyperparameters shown in Appendix B. 500 samples of the design space were generated with Latin hypercube sampling. After determining the hyper parameters, the surrogate model was passed into fmincon as an anonymous function. The active-set algorithm was used to find the optimum design.

Results

The first step in the optimization was to determine what span of τ to solve Equation 1 over. In order to inform this decision, a convergence study was conducted where the objective function was evaluated with the nominal design over a range of τ values. These results were then plotted vs the resulting standard deviation, and are shown in Figure 2. The results of this study indicate that any value above 5000 will give reasonable results, although values above 10000 are ideal. τ was chosen to be 7000 as a reasonable balance between performance and accuracy.

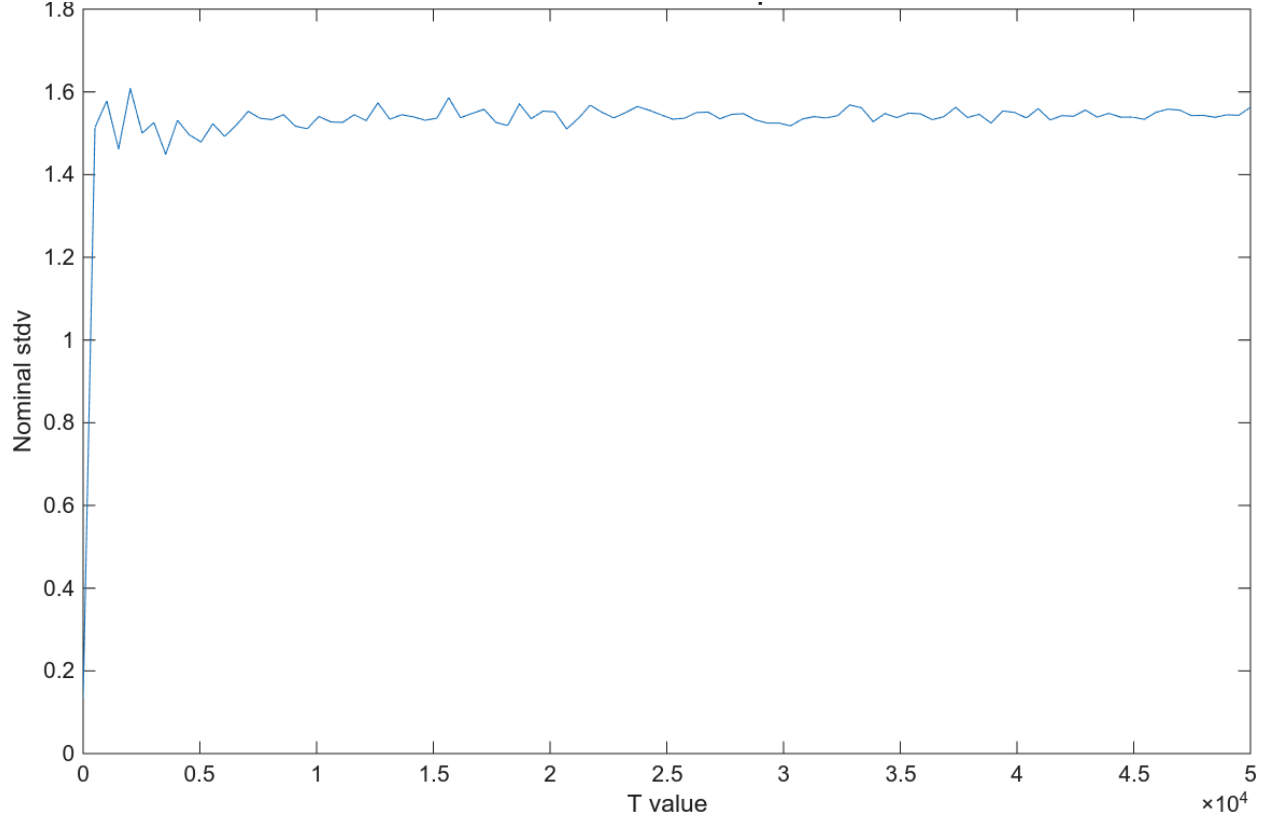


Figure 2: Convergence study.

Once τ was determined, the design space could be sampled and a surrogate model could be created. In order to evaluate the quality of the surrogate, a slice of the actual design space was plotted over the surrogate in order to evaluate if the general trends were being sufficiently captured. Creating this representation involved holding $\alpha_1 = 0.058$ and $r_2 = 0.8m$, and allowing ω to vary between 0.3 and 1. Samples from the objective function were then plotted along with evaluations of the surrogate model. The results of this exercise are displayed in Figure 3.

The results suggest the surrogate is fairly accurate, with a maximizer occurring at roughly $\omega = 0.7$. The sample data indicates that a maximizer may occur at roughly $\omega = 0.65$. The discrepancy in these values may be due to the fact that the surrogate is capturing higher order behavior due to the other design variables.

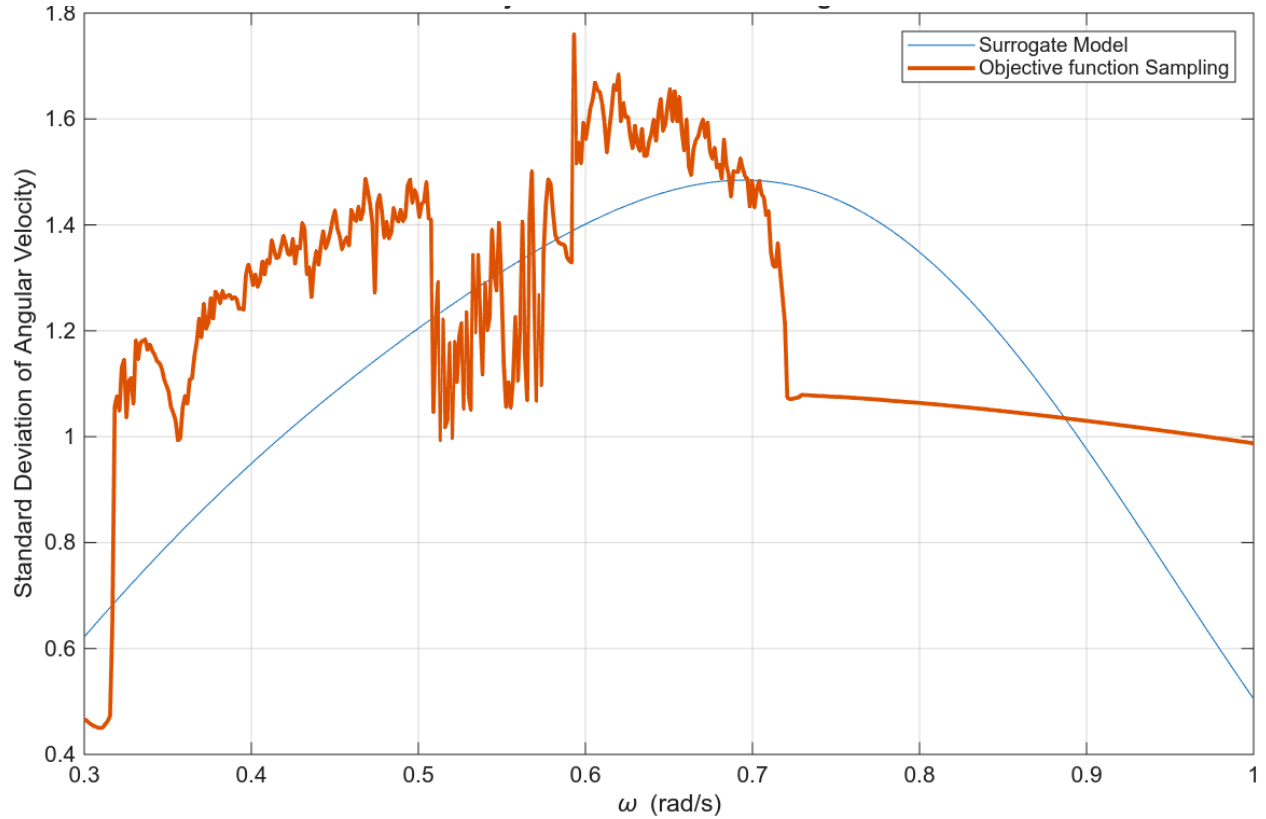


Figure 3: Quality of the surrogate.

With the quality of the surrogate verified, `fmincon` was run with the surrogate as an anonymous function. The results of the optimization are displayed in Figure 4. The minimization displayed in the objective value plot indicates successful minimization of the objective function. Additionally, first order optimality approaching zero indicates that this area of minimization is likely a minimizer. Finally, feasibility approaching zero indicates that none of the constraints are being violated, and therefore the optimal design lies in the feasible space. The optimized design variables are shown in Table 1 and the resulting standard deviation of angular velocity is compared to that of the nominal design.

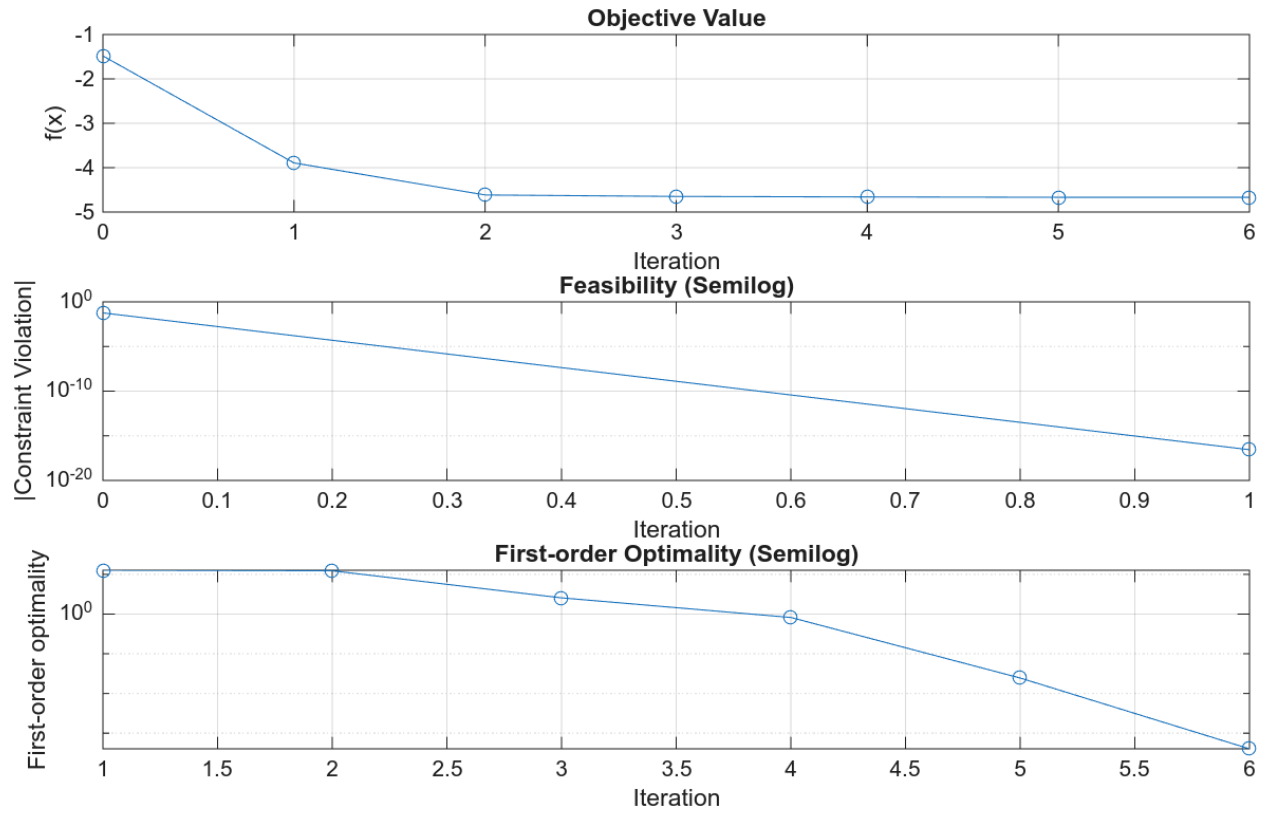


Figure 4: Evidence of convergence.

Table 1: Optimized design variables

	ω (rad/s)	α (rad)	r (m)	σ
Nominal Design	0.6807	0.058	0.8	1.5203
Optimized Design	0.7028	0.3	0.1574	5.2156
Improvement	343.06%			

Conclusions

The optimization of the Tilt-A-Whirl ride successfully achieved the goal of maximizing the standard deviation of angular velocity, thereby enhancing the unpredictability and excitement of the ride's motion. Using a Gaussian Process surrogate model and MATLAB's `fmincon` solver, the design variables converged to feasible values that produced a 343% improvement in the

objective function relative to the nominal design. The convergence data showed first-order optimality approaching zero and no constraint violations, confirming that the optimizer found a valid local minimum within the feasible space.

While the optimization results were successful, the underlying assumptions introduce limitations that should be considered. Treating each car as a point mass and neglecting its rotational inertia likely led to an overestimation of angular velocity variability. Furthermore, the correlation between angular velocity variation and rider enjoyment remains qualitative. Future work could improve the realism of the model by incorporating mass distribution and passenger effects, or by adding comfort and safety constraints to balance performance and rider experience.

Appendix

A. References

[1] Project handout

[2] Tilt-A-Whirl paper

B. Surrogate options

```
covfunc = @covSEiso;
likfunc = @likGauss;
sn = 0.1; % noise level
hyp.lik = log(sn);
hyp.cov = [log(0.3); log(0.75)]; % 1 = 0.3 sigma = 0.75
hyp.mean = []; % mean is 0
hyp = minimize(hyp, @gp, -100, @infExact, [], covfunc, likfunc, X, sample);
```

C. runopt.m

```
% This file runs the optimization algorithm
% set path for gpml
mydir =
'/home/quintenpienaar/src/MATLAB/NumericalDesignOptimization/Project3/gpml-matlab-v3.6-2015-07-07/';
addpath(mydir(1:end-1))
addpath([mydir, 'cov'])
addpath([mydir, 'doc'])
addpath([mydir, 'inf'])
addpath([mydir, 'lik'])
addpath([mydir, 'mean'])
```

```

addpath([mydir, 'prior'])
addpath([mydir, 'util'])

% Sample obj with LHS
N = 500; % number of samples
dv = 3; % number of design variables
X = lhsdesign(N, dv);

% Scale dvs
X(:,1) = scale(X(:, 1), 0.314159, 0.837758); %omega
X(:, 2) = scale(X(:, 2), 0, 0.3); %alpha
X(:, 3) = scale(X(:, 3), 0.2, 1.5); %r2

% T TBD
T = 7000;

% Generate samples
sample = zeros(N, 1);

for i=1:N
    sample(i, 1) = obj(X(i, 1), X(i, 2), X(i, 3), T); % Obj samples
    fprintf('Sample %d out of %d processed.\n', i, N);
end

close all

% Create surrogate
covfunc = @covSEiso;
likfunc = @likGauss;
sn = 0.1; % noise level
hyp.lik = log(sn);
hyp.cov = [log(0.3); log(0.75)]; % 1 = 0.3 sigma = 0.75
hyp.mean = []; % mean is 0
hyp = minimize(hyp, @gp, -100, @infExact, [], covfunc, likfunc, X, sample);

% test surrogate
z = [0, 0.058, 0.8];

```

```

w = linspace(0.3, 1, 100);    % 10 points from 0.3 to 1
y = zeros(size(w));           % preallocate y
for j = 1:length(w)
    z(1, 1) = w(j);
    y(j) = gp(hyp, @infExact, [], covfunc, likfunc, X, sample, z);
    fprintf('Surrogate sample %d out of %d processed.\n', j, length(w));
end
plot(w, y);
hold on
run('testingObj.m');
xlabel('\omega (rad/s)')
ylabel('Standard Deviation of Angular Velocity')
title('Objective Function and Surrogate')
legend('Surrogate Model', 'Objective function Sampling')
grid on
%% run optimization
% create lb and ub order is omega, alpha, r2
lb = [0.314159; 0; 0.1];
ub = [0.837758; 0.3; 1.5]; % upper bounds for omega, r2, alpha
z0 = [6.5*2*pi/60, 0.058, 0.8]; % initial guess for optimization - nominal
values for now - should be a row vector
options = optimset('GradObj', 'off', 'GradConst', 'off', 'Display', 'iter',
'Algorithm', 'active-set'); % not providing gradient of objective or
constraints
[x, fval] = fmincon(@(z)-gp(hyp, @infExact, [], covfunc, likfunc, X, sample,
z), z0, [], [], [], [], lb, ub, [], options);

```

D. obj.m

```
function [f] = obj(w, alpha1, r2, T)

% This function calculates the objective function

% Inputs:

%   w: omega, the

%   alpha1:

%   r2:

%   T = span of solution space

% Outputs:

%   sigma: The standard deviation of angular velocity dphi/dt

% define gamma, epsilon, alpha, beta

gamma = (1/(3*w))*((9.81/r2)^0.5);

Q0 = 20;

epsilon = 4.3/(9*r2);

% Initial conditions are [0, 0]

x0 = [0;0];

% Note we are in tau space not time space

[tau, x] = ode45(@(t, x) rhs(t, x), [0 T], x0);

% Find standard deviation

ylbar = (1/T)*(trapz(tau, x(:,2)));

integrand = (x(:, 2) - ylbar).^2;

I = trapz(tau, integrand);

f = 3*w*sqrt((1/T)*I);

function xdot = rhs(tau, x)

    % This function defines the state variables in ODE 27

    % Inputs:

    %   x: A vector containing states. x(1) = initial angle, x(2) = initial
angular velocity
```

```

% Outputs:

%   xdot: First derivative of x

alpha = 0.036 - (alpha1*cos(tau));

beta = 3*alpha1*sin(tau);

xdot = zeros(2, 1);

xdot(1) = x(2);

xdot(2) = -(

((gamma/Q0)*(x(2)))+(epsilon-(gamma^2*alpha))*(sin(x(1))))+(((gamma^2)*beta)*(

cos(x(1)))) );

end

end

```

E. determineT.m

```

% This script determines the optimal value of T

% Simulate nominal design for a range of T

w = 6.5*2*pi/60;

alpha = 0.058;

r = 0.8;

T_values = linspace(1, 50000, 100); % Define a range of T values for simulation
performanceMetrics = zeros(size(T_values)); % Preallocate array for performance
metrics

for i = 1:length(T_values)

    T = T_values(i);

    % Calculate performance metric for each T (example calculation)

    performanceMetrics(i) = obj(w, alpha, r, T);

    fprintf('Objective sample %d out of %d processed.\n', i, length(T_values));

end

plot(T_values, performanceMetrics);

```

F. fmincon_plots.m

```
% Optimization iteration data

Iter = [0 1 2 3 4 5 6]';

Fcount = [4 8 14 20 24 28 32]';

fval = [-1.48238 -3.89359 -4.61533 -4.64812 -4.65953 -4.66789 -4.6679]';

MaxConstraint = [-0.058 2.776e-17 0 0 0 0 0]';

StepLength = [NaN 1 0.25 0.25 1 1 1]'; % NaN for undefined initial step

DirectionalDerivative = [NaN -6.92 -9.72 -2.44 -1.72 -1.15 -0.025]';

FirstOrderOptimality = [NaN 12.7 12.3 2.56 0.821 0.0245 0.000403]';

%% Objective and optimality plots

figure;

% Objective value

subplot(3,1,1);

plot(Iter, fval, '-o');

xlabel('Iteration'); ylabel('f(x)');

title('Objective Value');

grid on;

% Feasibility (constraint violation)

subplot(3,1,2);

semilogy(Iter, abs(MaxConstraint), '-o');

xlabel('Iteration'); ylabel('|Constraint Violation|');

title('Feasibility (Semilog)');

grid on;

% First-order optimality

subplot(3,1,3);

semilogy(Iter, FirstOrderOptimality, '-o');

xlabel('Iteration'); ylabel('First-order optimality');

title('First-order Optimality (Semilog)');
```

```

grid on;

%% Separate constraint violation plot (optional)

figure;

plot(Iter, MaxConstraint, '-o');

xlabel('Iteration'); ylabel('Constraint Violation');

title('Constraint Violation (Linear Scale)');

grid on;

```

G. testingObj.m

```

% Define parameter constants

alpha1 = 0.058;    % example value, adjust as needed

r2 = 0.8;          % example radius value

T = 7000;

% Create omega range (0 to 1 linearly spaced)

w = linspace(0.3, 1, 500);

fvals = zeros(size(w)); % preallocate

% Evaluate f(w) for each omega

for i = 1:length(w)

    if w(i) == 0

        fvals(i) = NaN; % avoid divide-by-zero at w = 0

        fprintf('Calculation %d skipped. \n', i);

    else

        fvals(i) = obj(w(i), alpha1, r2, T);

        fprintf('Objective sample %d out of %d processed.\n', i, length(w));

    end

end

% Plot results

plot(w, fvals, 'LineWidth', 1.8)

```

H. scale.m

```
function y = scale(x, lb, ub)

% This function scales a design variable

% Inputs:

%   ub - Upper bound of design variable
%   lb - Lower bound of design variable
%   x - An array containing the values to be scaled

% Outputs:

%   y - An array containing the scaled DV

y = x .* (ub - lb) + lb;
```